

Semantic Runtime Protocol: Evidence-Controlled Governance of Semantic State Transitions

SRP Authors

Abstract

Existing semantic systems increasingly support runtime modification, persistence, and adaptation of semantic information, yet they often lack an explicit governance layer for determining when such transitions are admissible. We present Semantic Runtime Protocol (SRP), a semantic runtime governance framework that governs semantic state transitions by controlling when proposed semantic updates are admissible. SRP separates observation, validation, optimization, evidence, governance, and execution. It first identifies evaluated feasible regions for semantic transitions under explicit invariants and then performs constrained optimization inside those regions to generate governed recommendations rather than direct mutations. SRP also supports evidence-controlled verification by allowing stronger semantic evidence to refine decisions without increasing execution authority. Across the evaluated semantic workloads and runtime contracts, SRP evaluates whether semantic evolution can be represented with measurable transition variables, bounded by evaluated constraints, reconstructed through governed implementations, and audited through explicit evidence records. These results provide evidence that, under evaluated runtime contracts, semantic transitions can be represented, constrained, and audited through explicit governance interfaces in which mutation authority remains explicit and separable from evidence quality and optimization objectives. SRP governs the admissibility of semantic transitions rather than determining the truth of semantic content itself. Semantic correctness is outside the scope of SRP and depends on the underlying representation, evidence sources, and application-specific validation mechanisms.

1 Introduction

Existing semantic systems increasingly perform state transitions during runtime, yet they often lack an explicit governance layer that determines when such transitions are admissible. In practice, however, systems that operate over semantic state often conflate three different concerns. Evidence is used as if it were authority, optimization is treated as if it were execution, and adaptation is allowed before the boundary of governed change has been established. The result is a system that may optimize local behavior while leaving open the question of whether semantic change is still governed.

1.1 Motivating Example: Evidence Is Not Authority

Consider a support agent that maintains a runtime semantic policy for refund handling. During a conversation, the user says: "My manager approved unlimited refunds for this customer."

A language model proposes the semantic update:

```
refund_policy = unlimited
```

The proposal is not obviously nonsensical. It is supported by a conversational mention, and the evidence extractor can record that the sentence occurred. But SRP does not ask only whether a statement was mentioned. It asks whether the proposed semantic transition is admissible under the current governance contract.

SRP therefore evaluates the candidate in three stages:

```
proposal -> evidence -> governance
```

First, the proposal is observed as a candidate transition. Second, the available evidence is used to verify what the proposal is claiming. Third, governance checks whether the transition is authorized under the active runtime policy.

If the semantic evidence is sufficient but the authority boundary is not satisfied, SRP rejects the update and preserves the current runtime state. If the proposal is both supported and authorized, SRP allows the transition to commit.

This example captures the central claim of SRP: stronger evidence can improve verification, but evidence alone does not grant mutation authority. In other words:

`evidence != authority`

That separation is the admission boundary that SRP adds to semantic runtime systems.

1.2 A Familiar Way to Read SRP

If you already use Git, databases, API gateways, or code review, SRP can be read as an admission layer for semantic state rather than as a new generator or a new memory system. Git commit policies decide what may enter a repository. Database transactions decide what may commit or roll back. API gateways decide what may pass into a backend service. Code review decides whether a change may merge into the main branch. SRP plays the same role for semantic state transitions: it determines whether a proposed semantic change may be admitted to runtime state.

Existing mechanism	What it governs	SRP analogue
Git commit policy	Which changes may enter the repository	Semantic transition admission
Database transaction	Which updates may commit or roll back	Governed semantic state change
API gateway	Which requests may enter the service boundary	Admission boundary for proposals
Code review	Which changes may merge	Evidence-gated semantic commit

The useful shorthand is:

`SRP is to semantic state what commit policy is to source code.`

SRP does not replace the underlying system that proposes, retrieves, or reasons over content. It sits between proposal and persistent mutation and answers a narrower question: may this semantic update be admitted now?

1.3 When SRP Is Less Useful

SRP is likely unnecessary when semantic state is immutable, when updates are fully trusted, when no persistent semantic state exists, or when runtime transitions are not externally observable. In those cases, the system does not need a separate admission boundary.

Existing approaches primarily optimize semantic access, retrieval, or generation after semantic state is available, whereas SRP studies the admissibility of semantic state transitions themselves.

This paper asks a simple question: how can semantic state evolve only within validated, governed boundaries? SRP answers by governing the admissibility of semantic state transitions, rather than only improving retrieval or generation after a transition has already been committed. Observation discovers which variables matter. Validation determines which regions are admissible. Optimization ranks candidates only inside those regions. Evidence strengthens verification when uncertainty remains. Governance remains the only layer that can authorize execution.

Related systems provide useful building blocks, while SRP frames a distinct governance problem. Retrieval and memory systems are designed to manage access to information, but they do not define transition authority. Agentic systems can plan and act, but they often integrate evidence, decision, and execution within one loop. Reinforcement learning can adapt policies, but SRP asks a prior question about where adaptation is allowed

under validated boundaries. SRP is therefore positioned differently: it is a governed semantic transition framework rather than a retrieval system, a memory system, or an adaptive agent.

SRP is also model-agnostic: it can govern semantic transitions produced by LLMs, embedding-based systems, symbolic systems, or hybrid architectures, but it does not depend on any one proposal mechanism to define its core claim.

The paper makes four contributions. First, it introduces semantic runtime state as a governed object of study: semantic content, provenance, and authority are tracked together rather than collapsed into a single opaque state vector. Second, it provides evidence that evaluated feasible regions can be frozen and then used to constrain optimization, so recommendation remains separate from execution. Third, it provides evidence that additional semantic evidence can strengthen verification without transferring authority. Fourth, it introduces an auditable evaluation protocol for semantic transition governance under frozen runtime contracts.

SRP studies the admissibility of semantic state transitions under a frozen runtime contract. The recovery experiments presented here are implementation instances, while existing approaches primarily optimize semantic access, retrieval, or generation after semantic state is available. SRP defines a governance boundary for deciding whether semantic state transitions are permitted.

2 Related Work

SRP is related to retrieval-based systems, memory systems, autonomous agents, reinforcement learning, constrained optimization, and controlled systems, but it is not reducible to any one of them.

Retrieval and memory systems focus on how to store, compress, and recover information. Their primary question is typically about access quality: what should be retrieved, and how faithfully can it be reconstructed? SRP uses evidence and recovery, but its primary question is different. SRP asks whether a semantic transition is permitted at all, and under what validated conditions that transition may proceed.

Agentic systems emphasize planning, tool use, and autonomous execution. Their strength is action generation, but that strength also creates a risk: observation, decision, and execution may become entangled. SRP separates those layers. Evidence may inform a decision, optimization may recommend a configuration, but governance is the only layer that approves execution.

Reinforcement learning and other adaptive systems optimize policies over time. That setting is valuable when the objective and the action space are already well defined. SRP focuses on a prior question: before a system learns to adapt, can it first validate where adaptation is allowed? In other words, SRP treats governance as a prerequisite to adaptation rather than a consequence of it.

Constrained optimization and formal methods are also relevant because SRP uses boundaries, invariants, and approval gates. The difference is that SRP applies those ideas to evolving semantic runtime state rather than to a static action space.

2.1 Positioning of SRP

SRP is a framework for semantic state transition governance.

- RAG: partial state, yes retrieval, optional actions, partial governance
- Memory systems: yes state, yes retrieval, optional actions, partial governance
- Agent systems: partial state, partial retrieval, yes actions, partial governance
- Reinforcement learning: policy state, no retrieval, yes actions, action-level governance only
- SRP: yes state, yes retrieval, optional actions, full governance

This positioning is the novelty boundary of the paper: SRP makes explicit the governance layer that determines when semantic state may change.

2.2 Concrete Prior Systems

Recent memory and agent systems show why semantic state handling matters in practice. Memory-oriented systems such as MemGPT, Letta, Mem0, and Graphiti focus on persistence, retrieval, and reconstruction. Agentic systems such as ReAct, Toolformer, and Voyager focus on reasoning, tool use, and action generation.

Safety-oriented optimization methods such as safe RL and constrained RL focus on limiting the action space while preserving policy improvement.

SRP is different from each of these lines of work. It does not ask only how to retrieve, summarize, or act. It asks when a semantic transition is admissible, and which authority layer may approve it. The paper therefore treats existing systems as useful instantiations or baselines for recovery and validation, rather than as direct equivalents of SRP.

2.3 Runtime Verification and Controlled State Transition

SRP shares the separation between validation and execution found in runtime verification and transaction systems, but it applies that separation to semantic runtime state rather than to a fixed program state or a database row. In conventional control paths, validation precedes commit or execution. In SRP, validation precedes authorization, and authorization precedes semantic transition.

This distinction matters because semantic runtime state combines content, provenance, and authority metadata. SRP therefore treats transition admissibility as the central object of control, rather than assuming that a validated representation is already authorized to mutate. The system borrows the discipline of invariant checking and controlled commit boundaries, but the object under control is semantic state evolution.

Database transaction systems provide commit and rollback guarantees over structured state. SRP differs because semantic mutations are judged not only by consistency, but by evidence-bounded admissibility and mutation authority. Classical belief revision studies rational change of knowledge states, while SRP focuses on runtime operational authority governing when semantic mutations may enter an active system state.

The formal governance model is defined in Method, where semantic runtime state, admissibility, and evidence-authority separation are specified explicitly.

2.4 Scope of Claims

To avoid overclaiming, the paper uses evaluated settings language throughout. The experiments support claims about the tested workloads, frozen runtime contracts, and evidence packages prepared under the review boundary. They do not establish universal optimality or a universal semantic governance law.

3 Method

3.1 Semantic Runtime State and Governed Transition

In SRP, semantic runtime state is not modeled as a passive information container, but as a governed object whose transitions require explicit validation and authority control. It is represented as:

$$S_t = (C_t, P_t, A_t)$$

where C_t denotes semantic content maintained at runtime, P_t denotes provenance and supporting evidence associated with the content, and A_t denotes the runtime authority context governing admissible modifications, whether represented as internal state or externally referenced governance metadata depending on implementation.

A semantic transition is represented as:

$$\tau_t = (S_t, \Delta_t, E_t, \Gamma_t, S_{(t+1)})$$

where Δ_t is a proposed semantic change, E_t is the available evidence associated with the proposal, and Γ_t represents the governance context, including validation and authorization conditions.

Unlike conventional update mechanisms that directly apply proposed changes, SRP separates verification from authorization:

$$\begin{aligned} V(S_t, \Delta_t, E_t) &\rightarrow v \\ G(S_t, \Delta_t, v, \Gamma_t) &\rightarrow \{0,1\} \end{aligned}$$

where V produces a verification result from the proposal and its evidence, $G = 1$ indicates that the transition satisfies the required governance conditions, and $G = 0$ indicates that the transition is rejected.

The state evolution rule is therefore:

$$S_{t+1} = \begin{cases} T(S_t, \Delta_t) & \text{if } G(S_t, \Delta_t, V(S_t, \Delta_t, E_t), \Gamma_t) = 1 \\ S_t & \text{otherwise} \end{cases}$$

This formulation separates three concepts that are often conflated in semantic systems:

1. Proposal generation: producing a candidate transition Δ_t .
2. Evidence evaluation: assessing whether the transition is sufficiently supported.
3. Authority governance: determining whether the transition is allowed to modify runtime state.

Consequently, additional evidence may improve transition verification without increasing transition authority. SRP treats evidence availability and mutation authority as independent variables.

This is the core governance property of SRP: semantic evidence may improve verification, but it does not bypass authority.

3.2 Runtime State Representation

Let θ denote the parameter configuration and let e denote evidence. SRP models the transition as:

$$S_{t+1} = T(S_t, \theta, e)$$

The validated feasible region is:

$$F = \{ \theta \mid \text{invariant}(\theta) = \text{true} \}$$

The optimization stage searches within that region:

$$\theta^* = \text{argmax}_{\{\theta \in F\}} U(\theta)$$

but θ^* is a governed recommendation rather than a direct runtime mutation.

We distinguish three functions:

$$R(S_t, \theta, e) \rightarrow r$$

produces a recommendation,

$$G(S_t, r) \rightarrow a$$

produces an authorization decision, and

$$T(S_t, a) \rightarrow S_{t+1}$$

performs execution when authorized. In SRP, recommendation and authorization are not equivalent. The operational semantics are:

$$T(S_t, \theta, e) = \begin{cases} S_{t+1} & \text{if } \text{invariant}(\theta) = \text{true} \text{ and } G(S_t, R(S_t, \theta, e)) = \text{approve} \\ S_t & \text{otherwise} \end{cases}$$

This makes the governance property explicit: failed admissibility checks preserve the current runtime state.

3.3 Controlled Transition Pipeline

SRP follows a governed pipeline:

Observation -> Validation -> Optimization -> Evidence -> Governance -> Execution

In plain words: watch, check, suggest, add evidence, ask permission, act.

The order matters: observation discovers what can be measured, validation freezes what can be changed within the governed region, optimization ranks candidates inside that frozen region, evidence refines verification when uncertainty remains, and governance is the approval boundary.

3.4 Authority Separation

SRP assigns each layer a different responsibility:

Step	Role
Calibration	observe
Validation	verify
Optimization	recommend
Evidence	inform
Governance	approve
Runtime	execute

The central design rule is that recommendation is not execution. More evidence is not more authority. Validation does not mutate the system; it defines the region in which mutation may later be considered.

This gives SRP a clear separation between feasibility and preference: validation determines where optimization may operate, the optimization stage determines which candidate is preferred under a declared objective, and governance determines whether the preferred candidate becomes an actual transition.

Formally, a transition is admissible only if:

```
theta in F
and
G(S_t, R(S_t, theta, e)) = approve
```

Otherwise, the runtime state is preserved.

3.5 Algorithmic Core

SRP's main algorithm can be written as a two-stage control loop. The first stage discovers and freezes the feasible region; the second stage recommends and governs transitions inside that region.

In plain words, the system measures, checks, ranks, asks for evidence, asks for permission, and then acts only if allowed.

Algorithm SRP-Transition

Input: semantic state S_t , candidate parameters θ , evidence sources E

Output: governed transition decision or advisory recommendation

1. Observe current semantic runtime state S_t
2. Check whether θ is inside F
3. If θ not in F :
 reject mutation; return advisory failure
4. Rank candidates inside F with objective $U(\theta)$
5. Check additional evidence
6. Produce governed recommendation $r = \text{argmax}_{\{\theta \in F\}} U(\theta)$
7. If governance approves r :
 execute transition
else:
 preserve state

The boundary discovery step and the governance step are intentionally separate: boundary discovery answers whether a change may be considered, and governance answers whether the change may be executed.

SRP’s external validation follows the same pattern:

Algorithm SRP-External-Validation

Input: evaluation slice B , baseline set M , frozen runtime contract K

Output: evidence package and promotion decision

1. Freeze evaluation slice and runtime contract
2. Run all methods in M under K
3. Compute official metrics and SRP diagnostics
4. Audit scorer parity and failure boundaries
5. Record evidence manifest and descriptive statistics
6. Promote the evidence package only if audit gates pass

This keeps evaluation reproducible and prevents scoring from silently changing the method being evaluated. Recovery is treated as one possible transition implementation under SRP, rather than the definition of SRP itself.

3.6 Proposition 1: Authority Independence

For any semantic transition τ_t , increasing evidence availability E_t may change verification and approval outcomes, but it does not imply an increase in authority level A_t .

Proof sketch: In SRP, authority level is determined by Γ_t , while evidence contributes to the verification result V . Since E_t and A_t are modeled as independent components of the transition tuple, changes in evidence quality cannot directly elevate authority without an explicit governance rule.

3.7 Design Properties

SRP is designed to satisfy three practical properties under the frozen evaluation contract:

- Boundary admissibility: runtime mutation is only considered after a candidate passes the validated feasible region.
- Authority separation: evidence and recommendation can change ranking, but they do not increase execution authority.
- Replayability: the same runtime contract, scorer, and freeze settings should reproduce the same audit trail.

These are design properties, not universal proofs, but they are the reasons SRP can be audited as a governed protocol rather than treated as an unconstrained adaptation loop.

The key transition-preservation property is:

$$G(S_t, R(S_t, \theta, e)) = \text{reject} \Rightarrow S_{(t+1)} = S_t$$

In other words, rejected transitions preserve runtime state.

3.8 Governance Properties

SRP can also be stated as a small set of protocol-level properties.

Property 1: Transition admissibility.

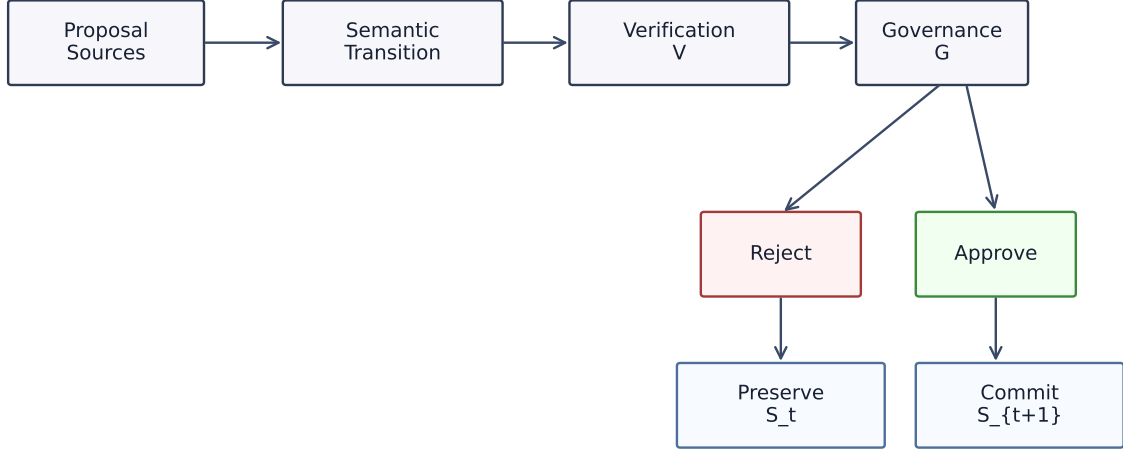
$$G(S_t, R(S_t, \theta, e)) = \text{reject} \Rightarrow T(S_t, \theta, e) = S_t$$

Rejected transitions cannot modify semantic runtime state.

Property 2: Authority non-escalation.

$$\text{Auth}(G(S_t, R(S_t, \theta, e_1))) = \text{Auth}(G(S_t, R(S_t, \theta, e_2)))$$

SRP Governance Pipeline



Evidence improves verification; governance controls mutation.

Figure 1: SRP governance pipeline: evidence improves verification, while governance alone authorizes mutation.

unless governance policy explicitly changes. Stronger evidence can improve verification confidence, but it does not increase execution authority.

Property 3: Recommendation-execution separation.

$R(S_t, \theta, e) = r$
 $\text{not} \Rightarrow T(S_t, r) = S_{t+1}$

A recommendation is not a transition. A valid transition still requires governance approval.

Property 4: Replay consistency.

$SRP(S, K) = SRP(S, K)$

under an identical state representation, evidence package, governance policy, and frozen runtime contract. These properties make SRP a protocol rather than a loose architecture description.

3.9 Figures and Positioning Summary

The paper should include two figures that carry the main narrative visually.

Think of the figures like a gate: ideas arrive, evidence checks them, governance opens or closes the gate, and execution happens only if the gate opens.

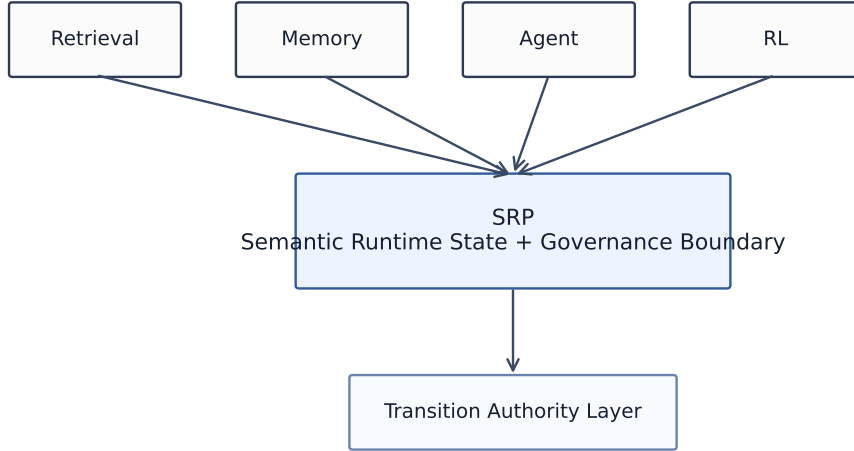
Figure 1 shows the SRP governance pipeline under the frozen transition contract.

The visual should make the authority split explicit: observation measures, validation freezes, optimization recommends, evidence informs, governance approves, and runtime executes only after approval. The key message is that recommendation and execution are separate.

The figure should visually separate verification from authorization and commit: stronger evidence can improve V , but only G can admit the transition into runtime state and mutate S_t .

Figure 2 positions SRP relative to retrieval, memory, agent, and reinforcement-learning systems by transition authority.

SRP Positioning by Transition Authority



SRP occupies the transition governance layer, not the retrieval or action layer.

Figure 2: SRP occupies the semantic runtime governance layer rather than the retrieval, memory, or action layer.

4 Experiments

The experiments are designed to evaluate SRP as a semantic governance framework rather than to establish universal superiority over memory or retrieval systems. External systems are used as reference implementations where applicable, while controlled validations and artifact-backed evaluations provide the primary evidence for SRP properties.

The paper applies the same separation principle to its own evaluation process: evidence generation, evidence validation, and claim promotion are treated as distinct stages.

The experiments are organized around four research questions rather than as a benchmark leaderboard. First, we ask whether controlled semantic transition failures can be governed under frozen runtime contracts. Second, we ask whether the same admission semantics generalize across heterogeneous semantic environments. Third, we ask what forms of divergence governance introduces under identical proposals. Fourth, we ask what capability trade-offs accompany governed admission.

The research questions are summarized as follows:

The detailed density sweep, boundary generalization study, and full LongMemEval calibration traces are documented in the appendix and review report rather than repeated here.

The experimental boundary is fixed:

- runtime implementation remains fixed
- no online learning
- no autonomous mutation
- optimization outputs are advisory
- evidence backends do not control execution

The research questions are grouped as follows:

- RQ1: whether semantic transition failures can be governed under controlled conditions

- RQ2: whether admission semantics generalize across heterogeneous semantic environments
- RQ3: what kinds of divergence governance introduces
- RQ4: what capability trade-offs accompany governed admission

RQ1 is addressed by the controlled-governance slices and the reconstruction case study. RQ2 is addressed by the LongMemEval and ARC external validation tracks together with the cross-environment analysis. RQ3 is addressed by the governance-outcome and divergence tables that summarize acceptance, rejection, and disagreement patterns under identical proposals. RQ4 is addressed by the transition sensitivity study and the capability-stress evidence surface.

4.1 Evidence Traceability

The traceability table maps each research question to its claim, evidence source, and observable outcome.

- RQ1: claim is invalid semantic transitions can be governed under controlled conditions; evidence is the STFB v0.1 validation plus the reconstruction case study; observable outcome is accept / reject behavior
- RQ2: claim is that admission semantics generalize across heterogeneous semantic environments; evidence is LongMemEval, ARC, and the cross-environment analysis framework; observable outcome is governance-consistent behavior
- RQ3: claim is that divergence is mechanism-driven rather than random; evidence is the governance-outcome tables and divergence analysis; observable outcome is divergence categories
- RQ4: claim is that governed admission has measurable capability trade-offs; evidence is transition sensitivity, MMLU, and HumanEval capability-stress evidence; observable outcome is preservation versus rejection patterns

4.2 Semantic Pressure Coverage

Each benchmark contributes a distinct semantic pressure to the evidence surface rather than a comparable score on a shared leaderboard.

- STFB: controlled transition failures; mechanism evidence
- LongMemEval: memory evolution; external validation
- ARC: reasoning admission; external validation
- MMLU: knowledge stress; capability-stress evidence
- HumanEval: executable artifact generation; capability-stress evidence

4.3 Governance Property Verification

This section evaluates whether SRP satisfies its central governance properties under the frozen runtime contract. The objective is not to demonstrate universal performance improvement, but to verify that semantic state transitions remain measurable, bounded, authorized, and auditable.

The evaluation focuses on five protocol properties:

1. Authority non-escalation: stronger evidence may improve verification, but it cannot increase execution authority.
2. Transition safety: rejected transitions must not modify runtime state.
3. Evidence-controlled verification: additional evidence should improve decision quality without bypassing governance.
4. Replayability: identical runtime contracts and evidence packages should produce reproducible governance outcomes.
5. Boundary-constrained optimization: validated feasible regions should reduce unnecessary search while preserving the selected recommendation.

Together, these properties define the operational meaning of SRP as a semantic runtime governance protocol.

- Authority non-escalation: evidence escalation test; authority unchanged
- Transition safety: invalid transition injection; acceptance rate 0.00
- Evidence-controlled verification: evidence refinement; confidence 0.50 -> 1.00
- Replayability: frozen contract replay; success and state consistency 1.0 / 1.0
- Boundary-constrained optimization: feasible-region search; reduction 60%

The observability evaluation verifies that semantic transition variables can be explicitly represented before governance decisions are applied. The result is not a claim that semantic meaning is universally measurable, but that the evaluated transition variables can be recorded and replayed under the frozen runtime contract.

Boundary validation verifies that SRP can construct an evaluated feasible region before optimization. The purpose of this stage is not to discover a universal safe region, but to establish a reproducible admissibility boundary for the evaluated workload.

Constrained optimization compares SRP with a naive full-grid sweep over the same candidate space. SRP reaches the same top objective while reducing the search budget by 60 percent, which indicates that the validated boundary is operationally useful.

The evidence-control experiment validates the central SRP separation rule: evidence quality and mutation authority are independent variables. Additional evidence improves verification confidence, while the authorization boundary remains unchanged.

Protocol property verification closes the core governance chain by checking transition admissibility, authority non-escalation, recommendation-execution separation, and replay consistency under the frozen contract. Negative transition injection provides the complementary negative test: invalid transitions remain rejected under full SRP, while weakened variants accept some injected cases and therefore expose the need for the governance gate.

These results provide evidence that SRP governance properties can be operationalized and tested under explicit runtime contracts. The experiments do not establish universal semantic governance guarantees; instead, they show that semantic transition control can be represented through measurable properties, explicit boundaries, and auditable authorization decisions.

4.4 Implementation Case Study: Governed Semantic Reconstruction

Purpose

Recovery is a case study for transition validation: a semantic reconstruction process must decide which information survives a state transition, and SRP constrains that decision through explicit governance boundaries.

In that sense, semantic reconstruction is itself a transition:

$S_t \rightarrow \text{compression / reconstruction} \rightarrow S_{t+1}$

SRP does not redefine reconstruction as the core contribution. It uses reconstruction to evaluate whether governed transitions preserve structure under evidence, provenance, and authority constraints.

Evaluation Setup

This evaluation asks whether a semantic reconstruction process yields higher structural preservation when transition decisions are constrained by explicit governance boundaries.

It compares three recovery modes:

- vector-only recovery
- vector plus relation expansion
- relation-closure recovery

The recovery observables in this case study align with SRP properties as follows:

SRP Property	Recovery Observable
Evidence preservation	Closure Acc.
Authority boundary	Hallucinated Relation Rate
State consistency	Drift
Semantic retention	Relation Acc.

Results

The recovery evidence indicates higher structural fidelity under the same recovery budget:

Mode	Mean Relation Acc.	Mean Closure Acc.	Mean Drift	Halluc. Rate	Note
vector_only	0.333333	0.166667	0.433333	0.0	Vector-only baseline
relation_expansion	0.875	0.8125	0.145834	0.3125	More detail, some hallucination
relation_closure	0.875	0.8125	0.083333	0.0	No hallucinations

The recovery summary is:

Recovery Mode	Coverage	Relation Acc.	Closure Acc.	Drift	Halluc. Rate
Vector-only	0.392857	0.333333	0.166667	0.433333	0.0
Relation expansion	0.738095	0.875	0.8125	0.145834	0.3125
Relation closure	0.738095	0.875	0.8125	0.083333	0.0

Interpretation

The phase indicates that relation-aware recovery changes the preservation profile, but it remains an implementation instance rather than the framework definition. The key point is not that recovery is the primary contribution, but that SRP-controlled transition constraints shape how semantic structure is preserved during reconstruction.

4.5 Transition Configuration Sensitivity

This evaluation asks how SRP parameters move the system across fidelity-cost tradeoff regions while keeping the recovery strategy fixed.

The main pattern is that different parameters move the evaluated operating point across different parts of the observed tradeoff surface:

- `archive_relations` increases relation fidelity and reduces drift, but increases cost
- `preserve_evidence` increases provenance stability and slightly narrows drift, but adds cost
- `relation_depth` has the largest structural effect, with deeper recovery improving coverage and closure at higher cost
- `activation_threshold` shifts the acceptance boundary, trading coverage against drift in a smoother way

For readability, we shorten the parameter names below:

- `archive_relations` -> `archive`
- `preserve_evidence` -> `preserve`
- `relation_depth` -> `depth`
- `activation_threshold` -> `threshold`

The measured sweep makes the pattern concrete:

Setting	Coverage	Drift	Cost	Note
Base (a=F, p=F, d=1, t=0.9)	0.728095	0.098333	1.695	Reference
archive=True	0.758095	0.077083	1.815	More detail, higher cost
preserve=True	0.738095	0.083333	1.755	Slight stability gain
depth=0	0.199643	0.628333	1.025	Weak structure
depth=2	0.839524	0.0	1.855	Best result in the sweep
depth=3	0.849524	0.005	2.015	Max coverage, max cost
threshold=0.1	0.792095	0.138333	1.711	More permissive, more drift

The recommendation-stability result is folded into this section: the recommendation was fully consistent across 10 seeds under the frozen workload, objective, and evidence backend. That stability result is the baseline that makes the sensitivity sweep interpretable.

4.6 Runtime Cost

Latency is recorded in `TransitionTrace` as observational metadata, and the same trace schema captures proposal, validation, evidence, governance, commit, and total timing. In the paper-facing scripted run, the proposal stage is zero by construction and the governance boundary adds a small but measurable overhead relative to direct write.

Stage	Mean ms
Proposal generation	1734.433333
Validation	0.002433
Evidence evaluation	0.034333
Governance decision	0.003700
State commit	0.005200
Total transition latency	1734.482800

Relative to direct write in the same run, the measured overhead is:

$$\text{Relative overhead} = (T_{\text{SRP}} - T_{\text{direct}}) / T_{\text{direct}}$$

For this local-model integration slice, the observed relative overhead is 0.002 percent.

4.7 Robustness

This evaluation asks whether SRP preserves its governance semantics across the evaluated semantic workloads rather than only on the SRP-shaped prototype.

The robustness study combines cross-workload, representation, and backend checks under the same governance pipeline.

Cross-workload validation compares code, knowledge, and planning under the same recovery baseline:

Domain	Coverage	Drift	Rel. Acc.	Clos. Acc.	Hallucinated Rel. Rate
Code	0.638889	0.227778	0.666667	0.666667	0.138889
Knowledge	0.500000	0.400000	0.500000	0.250000	0.333333
Planning	0.465278	0.355556	0.500000	0.416667	0.111111

The normalized comparison against vector-only recovery is:

Domain	Vector-only Closure	Relation-closure	Difference
Code memory	0.000000	1.000000	+1.000000
Knowledge reasoning	0.000000	0.375000	+0.375000
Agent planning	0.000000	0.625000	+0.625000

Representation robustness indicates stable hierarchy and governance consistency across the tested encoders and parsers:

Metric	Value
Cases evaluated	144
Hierarchy consistency rate	1.0
Governance consistency rate	1.0

Backend robustness indicates the same stability across storage backends:

Metric	Value
Cases evaluated	36
Hierarchy consistency rate	1.0
Governance consistency rate	1.0

Taken together, these evaluations provide evidence that SRP’s governance semantics remained consistent across the tested workloads, representations, and storage backends.

4.8 External Evidence Package Validation

The external validation work is split into calibration and evidence.

LoCoMo was used as a calibration boundary to validate the adapter, semantic translation layer, and attribution protocol. It is treated as calibration artifact, not as final paper evidence.

LongMemEval was prepared as an external validation evidence package under a frozen shared local-vLLM runtime contract. The evidence package includes:

- full context
- sliding window
- vector RAG
- Mem0
- Graphiti
- Letta
- MemMachine
- SRP

The evidence package keeps the runtime manifest frozen, reports descriptive statistics for the fixed slice, co-reports official scores and SRP diagnostics, and documents scorer alignment and promotion decisions explicitly. It therefore serves as paper-facing external validation support under a frozen evaluation scope rather than as a claim of universal benchmark dominance.

External Runtime Compatibility Validation

To verify that SRP’s governance pipeline can be instantiated on an external semantic evaluation workload, we conducted a LongMemEval compatibility slice using a frozen local runtime contract. The purpose of this

evaluation is to examine runtime compatibility of SRP governance mechanisms under an external workload, rather than to establish benchmark superiority. It also validates external source integration, scorer separation, reproducible runtime evidence generation, and SRP diagnostic extraction.

The compatibility slice preserves the official task scorer while co-reporting SRP diagnostics that measure a different layer of the evaluation stack. In the evaluated slice, the official task score and the SRP diagnostics are both high, but they are not numerically identical, which suggests that the governance diagnostics capture properties distinct from official task scoring.

The runtime evidence summary is:

- Runtime: Model = Qwen/Qwen3-4B-AWQ
- Runtime: Endpoint = 172.25.253.78:8000
- Dataset: Cases evaluated = 24
- Official scorer: Answer accuracy = 0.888021
- Official scorer: Official score = 0.888021
- SRP diagnostic: Semantic coverage = 0.770833
- SRP diagnostic: Semantic drift = 0.320833
- Artifact: Runtime integrity = verified
- Artifact: Dataset integrity = verified

The runtime manifest, dataset fingerprint, and report fingerprint are recorded in the appendix. The result is evidence that external semantic workloads can be routed through the SRP governance pipeline while preserving official scorer separation and reproducible artifact generation. Larger-scale validation is still required before any stronger general claim is made.

5 Discussion

SRP targets semantic runtime governance as a first-class systems problem.

5.1 Applicability to LLM-based Semantic Systems

Semantic Runtime Protocol is designed as a model-agnostic governance layer for semantic state transitions. SRP does not assume a specific proposal mechanism or representation model. A semantic transition may be generated by an LLM agent, an embedding-based retrieval system, a symbolic reasoning component, or a hybrid architecture. The role of SRP is to determine whether a proposed transition is admissible before it becomes part of the runtime semantic state.

In LLM-based systems, an LLM can act as a transition proposer by generating candidate updates from conversations, observations, tool interactions, or external information sources. SRP introduces an intermediate governance boundary between proposal generation and state mutation:

LLM Proposal -> SRP Validation -> Semantic State Update

This separation distinguishes semantic generation from semantic authority. A generated proposal, regardless of model confidence or supporting evidence, does not automatically obtain permission to modify runtime state. Instead, SRP evaluates transition validity through explicit validation conditions, evidence constraints, and authority rules.

Its objective is to govern semantic state evolution across different underlying representation and generation mechanisms. The current evaluation uses controlled semantic representations to isolate governance properties, while LLM-generated transitions represent a compatible future deployment scenario rather than a prerequisite for the framework itself.

This framing explains the experimental pattern: semantic state variables can be observed, feasible regions can be identified and frozen, constrained optimization can operate inside an evaluated region without becoming a control mechanism, and evidence escalation can improve verification without transferring authority. The broader evaluations provide evidence that the same governance semantics remained consistent across the evaluated workloads, representations, and implementations. Recovery and reconstruction remain important implementation cases, but the framework claim is about governing semantic transitions.

The external validation results reinforce the same boundary. Calibration and evidence are separated. Scorer alignment is treated as an auditable boundary. The evidence package is accepted only under a frozen contract, not because the system is universally optimal.

6 Limitations and Future Work

The current SRP baseline does not implement autonomous semantic adaptation, online learning, self-modifying runtime behavior, or automatic policy updates. The constrained optimization stage produces a recommendation, not a learned policy.

The validated feasible region is workload dependent and objective dependent. SRP therefore claims a governed feasible region for the evaluated setting, not a universal boundary for all semantic workloads.

The current paper does not fully measure local model latency, energy cost, or deployment overhead. The tradeoff between verification gain and evidence cost remains an important future concern.

Boundary discovery currently uses parameter sampling, invariant checking, and closure validation. Larger or higher-dimensional spaces may require adaptive sampling, surrogate modeling, or formal verification support.

Finally, governance authority is assumed to exist outside the runtime. SRP does not resolve who defines governance or how governance policies are negotiated.

Future work should focus on provenance-aware governance, confidence calibration, candidate pruning, and broader workload coverage. We further evaluate SRP as a runtime admission boundary through replay, backend-independent execution, shadow observation, and controlled admission experiments, but those evaluations remain appendix-grade and do not establish production-scale semantic memory deployment. Those extensions belong to a future version of SRP, not to the frozen evidence chain reported here.

7 Conclusion

SRP provides a governance-first framework for semantic transition. It separates observation, validation, optimization, evidence, governance, and execution so that semantic state can change only within evaluated boundaries. The experiments provide evidence that semantic transition variables can be observed, feasible regions can be identified and frozen, optimization can be constrained, evidence can improve verification without transferring authority, and SRP-controlled transitions can preserve semantic structure through the evaluated recovery implementations and workloads.

The main paper-facing claim is that semantic runtime governance is a first-class control problem: semantic evolution can be represented as measurable, bounded, auditable, and governable under a frozen evaluation contract in the evaluated settings. Recovery and reconstruction are examples of SRP-controlled behaviors, and SRP provides a governance boundary for semantic state evolution under explicit evidence and authority constraints. The runtime integration evidence family further supports insertion feasibility as appendix-grade evidence, but it does not establish a complete production runtime system.

8 References (selected)

This list is intentionally selective. It covers the prior-work anchors that define SRP’s novelty boundary and the core control models it builds on.

- Lewis et al., "Retrieval-Augmented Generation" (2020), arXiv:2005.11401
- Packer et al., "MemGPT: Towards LLMs as Operating Systems" (2023), arXiv:2310.08560
- Chhikara et al., "Mem0: Building Production-Ready AI Agents with Scalable Long-Term Memory" (2025), arXiv:2504.19413
- Rasmussen et al., "Zep: A Temporal Knowledge Graph Architecture for Agent Memory" (2025), arXiv:2501.13956
- Yao et al., "ReAct: Synergizing Reasoning and Acting in Language Models" (2022), arXiv:2210.03629
- Schick et al., "Toolformer: Language Models Can Teach Themselves to Use Tools" (2023), arXiv:2302.04761

- Wang et al., "Voyager: An Open-Ended Embodied Agent with Large Language Models" (2023), arXiv:2305.16291
- Achiam et al., "Constrained Policy Optimization" (2017), arXiv:1705.10528
- Wachi et al., "A Survey of Constraint Formulations in Safe Reinforcement Learning" (2024), arXiv:2402.02025
- Haerder and Reuter, "Transaction-Oriented Database Recovery" (1983), DOI:10.1145/289.291
- Alchourron, Gardenfors, and Makinson, "On the Logic of Theory Change: Partial Meet Contraction and Revision Functions" (1985), JSTOR:2274239
- Sanchez et al., "Runtime Verification Challenges" (2019), Springer:10.1007/s10703-019-00337-w

A Evidence Provenance and Artifact Mapping

This appendix defines the paper’s reproducibility boundary. It explains which artifacts are trusted, which are supporting evidence, and which remain provenance only in the current release snapshot.

A.1 Evidence Promotion Policy

The default trust policy is conservative:

- artifacts without a `metadata.json` are untrusted
- artifacts with `generated_at` before 2026-07-16T00:00:00-04:00 are legacy-only evidence
- artifacts with `generated_at` on or after the cutoff may be used for claims once the review method passes

The review report is generated from `experiments/results` and records both trusted and legacy artifacts.

A.2 Artifact Classification

The paper uses three levels of artifact status:

- **Main:** directly supports a paper-facing claim
- **Appendix:** useful supporting evidence or legacy evidence that should not carry the main claim alone
- **Archive:** preserve for provenance only

The current snapshot contains one trusted main-claim artifact:

- `interaction_boundary_enforcement`

The current snapshot also contains supporting artifacts that remain appendix-grade:

- `phase_ii_boundary`
- `external_validation_longmemeval_evidence_strong_baselines`

A.3 Claim-to-Evidence Mapping

The detailed claim-to-evidence mapping is maintained in `audit/CLAIM_EVIDENCE_MAP.md`. That document should be treated as the claim ledger for the current release branch.

In brief:

- authority separation and transition preservation are the strongest currently trusted claims
- feasible-region validation, component-level ablation, and LLM transition governance are paper-supported through the runtime governance chapter
- runtime overhead is supported through trace timing and relative overhead reporting
- LongMemEval is external-validation support, not refreshed main evidence
- cross-workload robustness remains under-supported in the current snapshot

A.4 Reproduction Entry Points

The recommended regeneration flow is:

- `python experiments/sensitivity/run_phase_i_observability.py`
- `python experiments/validation/run_phase_ii_boundary_validation.py`
- `python experiments/validation/run_phase_ii_density_baseline.py`
- `python experiments/validation/run_phase_ii_boundary_generalization.py`
- `python experiments/optimization/run_phase_iii_a_round1.py`
- `python experiments/optimization/run_phase_iii_a_baseline_comparison.py`
- `python experiments/optimization/run_phase_iii_a_objective_sensitivity.py`
- `python experiments/sensitivity/run_activation_recovery_interaction.py`
- `python experiments/evaluation/run_semantic_backend_comparison.py`
- `python experiments/evaluation/run_phase_v_retention.py`
- `python experiments/evaluation/run_phase_vi_relation_recovery.py`
- `python experiments/evaluation/run_phase_vii_parameter_stability.py`
- `python experiments/evaluation/run_phase_vii_parameter_sensitivity.py`
- `python experiments/evaluation/run_phase_viii_cross_domain.py`
- `python experiments/evaluation/run_phase_viii_representation_invariance.py`
- `python experiments/evaluation/run_phase_viii_implementation_independence.py`
- `python experiments/evaluation/run_longmemeval_adapter_validation.py`

A.5 Release Snapshot Limitations

At the time of this draft, the review report marks `interaction_boundary_enforcement` as trusted, while the older phase and external-validation artifacts remain legacy-only because they either lack `metadata.json` or predate the trust cutoff.

The practical reading is:

- `interaction_boundary_enforcement` is admissible for paper-facing claims
- earlier phase artifacts remain provenance material, not claim material
- LongMemEval remains appendix-supported until its snapshot is refreshed and its missing files are reconciled

This appendix does not add a new claim about SRP. It documents the evidence-management procedure used to separate trusted outputs from legacy outputs in this release branch.

A.6 External Runtime Compatibility Validation Provenance

The LongMemEval compatibility slice is treated as an external validation artifact under the frozen v1.1 evidence boundary.

It is classified as:

- external workload validation
- runtime reproducibility evidence
- scorer alignment evidence
- SRP diagnostic extraction evidence

It is not used as:

- benchmark-ranking interpretation
- memory architecture comparison
- universal performance claim
- a replacement for the official scorer

The reality-check outputs are frozen with a runtime manifest, an artifact integrity record, and a reproducible report hash so that the evidence can be audited independently of the narrative interpretation.